



INSTITUTO DE
COMPUTAÇÃO

MC322 - Programação Orientada a Objetos
Instituto de Computação - Unicamp
Primeiro Semestre de 2026
Professor: Marcos M. Raimundo (mraimundo@ic.unicamp.br)

Proposta de Projeto Final: Ecossistema de Componentes Robustos

1 Objetivo

O objetivo deste projeto prático é consolidar a transição da programação baseada em classes para a engenharia de software baseada em componentes isolados e orientados a objetos. Os estudantes deverão projetar e implementar um sistema completo no qual o núcleo das regras de negócio permaneça imutável diante das pressões de mudanças externas. O foco central está na preservação da modificabilidade, na redução do custo de manutenção a longo prazo e no respeito estrito à blindagem lógica de domínios ricos.

2 Descrição

O sistema deverá representar uma aplicação funcional, robusta e aderente às necessidades de um domínio real de negócio escolhido pelo grupo (ex: e-commerce, sistemas bancários, logística, redes sociais, etc.).

A linguagem de desenvolvimento recomendada é **Java**, utilizando sua infraestrutura gerenciada (JVM), ecossistema de compilação/automação (Maven via `pom.xml`) e sistema de tipagem estática. Caso o grupo opte por outra linguagem de programação, esta deve, obrigatoriamente, dar suporte nativo a polimorfismo dinâmico, encapsulamento estrito e contratos de interfaces, devendo as diferenças arquiteturais serem profundamente justificadas no relatório técnico.

3 Diretrizes de Design e Tópicos Abordados

O design do sistema será avaliado com base em métricas qualitativas e estruturais discutidas ao longo do curso. Os seguintes conceitos devem ser explicitamente evidenciados no código e na documentação:

- **Alinhamento com o Acrônimo TRUE:** O código deve ser demonstradamente *Transparent* (consequências de mudanças óbvias), *Reasonable* (custo de mudança proporcional ao benefício), *Usable* (reutilizável em novos contextos) e *Exemplary* (diretrizes de design consistentes).
- **Encapsulamento Operacional e Modelo Rico:** Fica terminantemente proibida a implementação de modelos puramente anêmicos (classes compostas exclusivamente por estruturas `get` e `set`). Os objetos devem atuar como máquinas de estado que expõem comportamentos públicos. Deve-se respeitar o princípio imperativo *Tell, Don't Ask* (Diga, Não Pergunte), mitigando a *Inveja de Recursos/Funcionalidades*.
- **Muros de Fronteira e Contratos de Abstração:** O sistema deve ser desenhado como uma rede de colaboradores baseada em **Interfaces**. Interfaces devem atuar como escudos de abstração pura, separando o *Espaço do Problema* (o QUE o negócio faz) do *Espaço da Solução* (COMO a infraestrutura implementa). O acoplamento físico à implementação deve ser substituído pelo acoplamento lógico a contratos estáveis.

- **Gestão de Acoplamento e Coesão:** Os componentes devem mitigar a *Síndrome do Amigo Íntimo* (falso desacoplamento). Na escala de Myers, o grupo deve buscar o acoplamento de dados ou de controle, eliminando o acoplamento de conteúdo ou comum (evitar o uso indiscriminado de estados estáticos globais). Classes devem demonstrar alta coesão interna, evitando o antipadrão da *Classe Deus* (*God Class*) sem incorrer em fragmentação excessiva (*Cirurgia de Espingarda*).
- **Subtipagem e Limites da Herança:** O uso da palavra-chave `extends` deve ser restrito a situações de real especialização essencial (relação *É-UM*). A substituíbilidade deve ser garantida respeitando o **Princípio de Substituição de Liskov (LSP)**, evitando o uso de herança puramente para reaproveitamento sintático de código. Onde aplicável, deve-se preferir a composição e a delegação à herança.
- **Polimorfismo e Extensibilidade (Design Patterns):** O sistema deve estar fechado para modificações destrutivas e aberto para expansões (Princípio Aberto/Fechado - OCP). Para isso, o projeto deve incorporar o uso consciente de, no mínimo, **três Design Patterns do GoF** (ex: *Strategy* para variação de algoritmos, *Template Method* para preservação de invariantes na classe base, *Factory/Builder* para isolamento da gênese de objetos, entre outros). A aplicação dos padrões deve focar na eliminação de estruturas condicionais (`if/else` ou `switch`) complexas por meio de polimorfismo dinâmico.
- **Gestão Estruturada de Falhas (Filosofia *Fail-Fast*):** O sistema deve tratar suas exceções semanticamente de acordo com as regras de negócio. Exceções devem validar os parâmetros e os estados logo no início das operações, impedindo a propagação de erros silenciosos. O grupo deve implementar pelo menos **duas exceções customizadas**, diferenciando falhas de negócio recuperáveis (*Checked Exceptions*) de erros críticos de infraestrutura ou violação de contratos (*Unchecked Exceptions*).
- **Polimorfismo Paramétrico (Generics):** O sistema deve se proteger de heterogeneidades acidentais através do uso de tipos parametrizados, garantindo a preservação da identidade semântica estrita das entidades e evitando conversões perigosas em tempo de execução (*ClassCastException*).
- **Persistência de Dados e Interfaces Gráficas:** A camada de persistência de dados (leitura e gravação de arquivos) e a camada de interface com o usuário (GUI, Web ou Mobile) devem atuar como meros detalhes de infraestrutura, isoladas do núcleo de regras de negócio por meio de inversão de dependências.

4 Atividades e Entregáveis

O desenvolvimento do projeto compreende as seguintes atividades obrigatórias:

- **Relatório Técnico de Arquitetura (PDF):** Documento formal contendo:
 - O **link para o repositório Git público ou compartilhado** de forma destacada na primeira página.
 - Mapeamento do domínio do problema e justificativa semântica das entidades escolhidas.
 - **Diagrama de Classes UML completo** detalhando os relacionamentos (associação, agregação, composição, herança e realização de interfaces).
 - **Matriz de Justificativa de Design:** Seção dedicada a demonstrar onde e por que os princípios de POO (TRUE, *Tell Don't Ask*, LSP, OCP) e os Design Patterns foram aplicados, explicitando o problema original e o ganho arquitetural obtido.
 - Guia de execução rápida automatizado (instruções do Maven ou equivalente).
- **Repositório Git e Gerenciamento de Dependências:** Código-fonte limpo, estruturado em pacotes coesos (ex: segregando *model*, *infra*, *view*), utilizando tipagem estática e comentários em inglês com forte documentação técnica. O arquivo de configuração (ex: `pom.xml`) deve gerenciar as dependências de forma limpa.
- **Defesa Oral e Demonstração Técnica:** Apresentação com duração de 10 a 15 minutos focada exclusivamente na engenharia do software. O grupo deverá realizar uma demonstração ao vivo e defender as escolhas de diagramação, o fluxo do *Late Binding* executado pela JVM nas interfaces chaves e o isolamento do modelo de negócio diante de falhas.

4.1 Formato dos Grupos

Os grupos deverão ser compostos por, no mínimo, 3 e, no máximo, 4 alunos. Exceções deverão pedir análise específica para o professor.

4.2 Datas e Prazos Principais

- **Entrega do Projeto Final (Relatório + Repositório Git):** Até o dia 22/06/2026 via Google Classroom.
- **Envio dos Slides de Suporte da Defesa:** Até o dia 25/06/2026.
- **Banca de Defesa Oral:** Dias 23/06/2026 e 25/06/2026, as apresentações serão sorteadas no dia 22 até meio-dia.

5 Critérios Base de Avaliação

A avaliação do projeto não consistirá na mera verificação de funcionamento técnico, mas sim na qualidade estrita da engenharia de objetos:

- **Arquitetura de Objetos e Justificativa de Design (45%):** Avaliação da blindagem lógica do estado, nível de acoplamento entre os componentes, aplicação correta de interfaces, respeito aos princípios contratuais (LSP, Demeter, *Tell Don't Ask*) e correta aplicação dos *Design Patterns* para substituição de fluxos procedurais. Qualidade e precisão do diagrama UML.
- **Robustez, Qualidade de Código e Testabilidade (20%):** Código idiomático, aplicação correta de Generics, estratégia de exceções *Fail-Fast* e cobertura/relevância da suíte de testes sugeridos.
- **Complexidade de Domínio e Aderência de Escopo (20%):** Volume de regras de negócio tratadas de forma rica pelo sistema e a completude das funcionalidades propostas pelo grupo.
- **Defesa Oral e Domínio Técnico (15%):** Clareza na explicação da arquitetura do sistema por todos os membros e fluidez da demonstração prática.

A nota final do projeto será estritamente **individualizada**. O relatório deve conter uma seção explicitando a divisão detalhada de tarefas. Adicionalmente, o histórico de *commits* no repositório Git e o domínio técnico demonstrado durante as perguntas da defesa oral serão utilizados como balizadores para a atribuição da nota individual de cada membro.

6 Integridade Acadêmica e Uso Ético de IAs Generativas

- **Plágio de Software:** A cópia parcial ou integral de projetos de terceiros, códigos extraídos da internet ou repositórios de anos anteriores sem a devida atribuição resulta em nota zero imediata para todo o grupo, com abertura de processo disciplinar institucional.
- **Uso de Inteligência Artificial (ex: ChatGPT, GitHub Copilot):** Ferramentas de IA generativa são permitidas exclusivamente como aceleradores de produtividade sintática (ex: geração de código repetitivo de infraestrutura, configurações iniciais de telas ou escrita de massa de dados para testes).
- **A Autoria e o Entendimento:** A modelagem do domínio do problema, o desenho das interfaces, a distribuição de responsabilidades e as justificativas técnicas das tomadas de decisão devem ser de autoria intelectual **exclusiva do grupo**. Os alunos devem ser capazes de reescrever, alterar ou explicar profundamente qualquer linha de código contida no projeto quando solicitados na defesa oral. A detecção de código gerado por IA cujos conceitos subjacentes não sejam dominados pelos estudantes será tratada como violação de integridade acadêmica.